LINUX SYSTEMS & KERNEL FOUNDATIONS

Reference Guide for Kernel Architecture, Virtualization, and Device Filesystems

1. What is the Linux kernel?

The kernel is the core program of an operating system. It acts as the ultimate middleman between your computer's hardware (CPU, memory, hard drives) and the software applications you run (like a web browser or a terminal).

Its main jobs include:

- **Memory Management:** Deciding which program gets how much RAM and where.
- **Process Management:** Scheduling which tasks get to use the CPU and for how long.
- **Device Drivers:** Allowing software to talk to hardware components.
- **System Calls:** Providing a secure interface for applications to request hardware resources.

2. What is a kernel module?

A Kernel Module (often called a Loadable Kernel Module or LKM) is a piece of code that can be loaded into or unloaded from the kernel on the fly, without needing to reboot the system. It dynamic expands the capabilities of the running kernel. Common implementations include:

- **Device drivers:** (e.g., adding support for a new USB Wi-Fi adapter you just plugged in).
- **Filesystem drivers:** (e.g., enabling the system to read a specific disk format like NTFS or ext4).
- **System extensions:** (e.g., netfilter modules for firewalls).

3. Why should beginners use a VM for kernel experiments?

When you write or modify kernel-level code, a single mistake can crash your entire system (causing a "Kernel Panic"). Beginners should always use a Virtual Machine (VM) for several critical reasons:

- **Safety:** A crash or a rogue pointer in the kernel can corrupt your hard drive or lock up the system. In a VM, a crash only affects the isolated virtual environment, keeping your host computer completely safe.
- **Easy Recovery:** You can take a "snapshot" of your VM before running an experimental module. If you break something, you can restore it to a working state in seconds.
- **Debugging:** VMs make it much easier to attach external debuggers and monitor system logs from the host environment while the guest kernel is running.

4. What is the purpose of the `/proc` folder?

The `/proc` directory is a virtual filesystem (sometimes called `procfs`). It doesn't actually exist on your hard drive; the files inside it are generated on the fly by the kernel in system memory.

Its purpose is to act as an information window into the kernel and running processes:

- For example, if you look at `/proc/cpuinfo`, the kernel dynamically generates a text file showing your current CPU hardware details.
- Directories designated with numbers (e.g., `/proc/1234`) contain real-time details about the specific process running under that Process ID (PID).

5 & 6. Identifying files in the `/dev` folder

The `/dev` directory contains device files (or device nodes), which represent the physical and virtual hardware on the system as standard file interfaces. Here is how to identify your storage devices and input devices:

> **Storage Devices (Hard Drives / SSDs):**
>
> Look for files starting with `sd` or `nvme`:
>
> - `sda`, `sdb`, `sdc`...: These represent SCSI/SATA hard drives or SSDs. `sda` is the first drive, `sdb` is the second, etc.
> - `sda1`, `sda2`...: The numbers represent individual **partitions** on that specific drive.
> - `nvme0n1`: If you are using a modern NVMe M.2 SSD, it follows this naming convention (`nvme0` is the controller, `n1` is the namespace/drive).

> **Input Devices (Keyboard / Mouse):**
>
> Input devices are generally grouped inside a sub-folder or use generic input prefixes:
>
> - `/dev/input/`: This directory contains most of your input hardware. Inside, you will see files like `event0`, `event1`, etc., which capture raw data from your keyboard and mouse.
> - `mouse0` or `mice`: Often found inside `/dev/input/`, these specifically handle mouse coordinates and data streams.
> - `psaux`: A legacy file historically used for older PS/2 mouse interfaces.